

Глава 3. Практическая часть

3.1. Сетевое взаимодействие

В проекте Few Seconds – Many Deaths сетевое взаимодействие реализовано с использованием Unity 6, Netcode for GameObjects и Unity Transport. Архитектура построена на модульной системе, включающей управление сессиями, синхронизацию данных игроков и игровых состояний, а также интерфейс для взаимодействия с лобби. Это обеспечивает стабильную работу кооперативного режима, позволяя игрокам совместно проходить уровни и координировать действия. Модули взаимодействуют через событийную систему GlobalEventSystem и синглтоны, такие как SessionsManager и PlayerInfoData, что упрощает масштабируемость и тестирование.

3.1.1. Управление игровыми сессиями

Управление игровыми сессиями реализовано через класс SessionsManager, который отвечает за создание, поиск и подключение к игровым лобби, обеспечивая кооперативное взаимодействие игроков. В методе Awake класса SessionsManager инициализируется пул объектов (sessionPool) для динамического отображения списка сессий в интерфейсе, а также создается экземпляр SessionService, взаимодействующий с Unity Multiplayer Services. В методе Start выполняется анонимная авторизация через AuthenticationService.Instance.SignInAnonymouslyAsync, что позволяет игрокам подключаться без необходимости создания учетной записи.

Создание сессии осуществляется методом StartSessionAsHost, где задаются параметры лобби: режим игры (sessionMod), приватность (IsPrivate), максимальное количество игроков (maxPlayers) и пароль (Password). Метод использует SessionService.CreateSessionAsync для создания сессии с поддержкой релейной сети (WithRelayNetwork). После успешного создания сессии вызывается метод InitialiseSession, который сохраняет данные сессии в LobbySaver.instance.currentSession, активирует игровой интерфейс через

`SessionUI.SetUIState(UIState.InLobby)` и обновляет информацию о лобби (код сессии, пароль) с помощью `SessionUI.UpdateLobbyInfo`.

Подключение к существующим сессиям возможно двумя способами: по уникальному коду через `JoinSessionByCodeAsync` или по идентификатору сессии через `JoinSessionByIdAsync`. В первом случае игрок вводит код в поле `joinCodeInputField`, и метод `JoinSessionByCode` обрабатывает подключение. Во втором случае используется метод `JoinSession`, который проверяет наличие пароля у сессии (`HasPassword`) и, при необходимости, активирует интерфейс ввода пароля через `SessionUI.ShowPasswordPrompt`. Если пароль введен корректно, метод `JoinSessionWithPassword` вызывает `TryJoinSession` для подключения.

Обновление списка доступных сессий реализуется через корутину `UpdateSessionListCoroutine`, которая с интервалом `sessionUpdateInterval` запрашивает данные через `SessionService.GetSessionsList`. Полученные сессии отображаются в интерфейсе с помощью `SessionUI.UpdateSessionList`, где для каждой сессии указываются название, статус приватности и количество подключенных игроков. Интерфейс поддерживает функционал копирования кода и пароля (`CopyText`) и отмены ввода пароля (`CancelPasswordPrompt`), обеспечивая удобство взаимодействия.

3.1.2. Синхронизация данных игроков

Синхронизация данных игроков в проекте реализована через класс `PlayerDataSyncService`, который обеспечивает передачу информации о персонажах, их цветах и выбранных навыках между клиентами в кооперативном режиме. Этот процесс гарантирует, что все игроки имеют актуальные данные о состоянии друг друга, что критически важно для согласованного игрового процесса.

Основной компонент, `PlayerDataSyncService`, работает в связке с `PlayerInfoData`, который выступает синглтоном для хранения данных игроков, включая списки цветов (`ColorList`), персонажей (`HeroDataList`) и выбранных

навыков (SkillChoiceList). При подключении нового клиента метод OnNewClientConnected запускает корутину WaitForPlayerInfoDataInitialization, которая ожидает инициализации PlayerInfoData и вызывает SendDataToNewClient. Этот метод отправляет данные о персонаже (ChangeHeroNewClientRpc), цвете (ChangeColorNewClientRpc) и навыках через PlayerDataSkillManager.SyncSkillsForClient с использованием RPC и ClientRpcParams для таргетинга конкретного клиента.

Листинг 3.1 — метод синхронизации героев между игроками

```
[ClientRpc]
private void ChangeHeroNewClientRpc(int ID, int heroDataID,
ClientRpcParams clientRpcParams = default)
{
    playerDataService.SetHero(ID, heroDataID);
    GlobalEventSystem.SendHeroChanged();
}
```

Класс ChangePlayersListNetwork обрабатывает изменения персонажей и навыков. Метод ChangeHero получает идентификатор персонажа через GetHeroDataID и передает его через ChangeHeroRpc, синхронизируя выбор персонажа на всех клиентах. Аналогично, метод ChangeSkill обновляет выбранный навык через ChangeSkillRpc, используя индексы навыков и их вариаций. Эти RPC-вызовы инициируют события GlobalEventSystem (SendHeroChanged, SendSkillChanged), которые уведомляют другие системы об обновлениях.

Данные игроков хранятся в структуре PlayerInfoData, где PlayerCount отслеживает количество подключенных игроков, а PlayerIDThisPlayer идентифицирует текущего игрока. События PlayerLobbyUpdate и PlayerDataChanged обеспечивают обновление данных через метод UpdatePlayerNickList, который синхронизирует никнеймы и другие параметры.

Для предотвращения дублирования персонажей используется метод `HasDuplicates`, проверяющий уникальность выбранных героев перед стартом игры.

Синхронизация выполняется в авторитарной модели, где хост управляет критическими данными, а клиенты получают обновления через RPC. Это минимизирует задержки и обеспечивает стабильность, особенно при динамическом подключении или отключении игроков, обрабатываемых через `ConnectPlayer` и `DisconnectPlayer` в `PlayerInfoData`. Такой подход гарантирует целостность игровых данных и их согласованность на всех клиентах, что является основой для кооперативного взаимодействия в игре.

3.1.3. Сетевая синхронизация игровых состояний

Сетевая синхронизация игровых состояний в *Few Seconds – Many Deaths* обеспечивает согласованность игрового процесса между всеми клиентами, включая позиции персонажей, стадии боя и действия игроков. Основной задачей является поддержание целостности игровых данных в кооперативном режиме, что достигается с использованием технологий `Netcode for GameObjects` и `Unity Transport`. Синхронизация реализована через комбинацию компонентов `ClientNetworkTransform`, `NetworkVariable` и RPC (Remote Procedure Calls), что позволяет передавать данные о перемещениях, навыках и очередности ходов между хостом и клиентами.

Перемещение персонажей по клеточному полю осуществляется с использованием класса `MapClass`, который строит пути на основе алгоритма `Pathfinding`. Данные о перемещении передаются хостом через RPC, вызываемые в `PlayerMovementController`. Метод отправляет ID игрока и список клеток пути, после чего клиенты выполняют соответствующее перемещение, синхронизированное через `ClientNetworkTransform`. Это гарантирует, что позиции персонажей одинаковы на всех клиентах, исключая рассинхронизацию.

Синхронизация стадий боя (предсказание, выбор действий, выполнение действий игроков, выполнение предсказания босса) реализована через

CombatStageManager и NetworkVariable. NetworkVariable хранит текущую стадию боя, автоматически синхронизируя её между хостом и клиентами. Переход между стадиями инициируется хостом, который уведомляет клиентов через RPC, обеспечивая одновременное обновление игрового состояния. Например, в стадии выбора действий игроки используют PlayerSkillManager для передачи информации о выбранных навыках (ID игрока, индекс навыка, координаты каста) через метод CastActionRpc. Эти данные отправляются хосту, который координирует их выполнение и рассылает результаты клиентам.

Очередность ходов управляется классом TurnOrderManager, который случайным образом определяет порядок действий игроков в начале каждого цикла. Метод RandomiseTurnOrder использует алгоритм перемешивания (System.Random) для генерации новой последовательности, которая синхронизируется через RPC SetNewTurnOrderRpc. Этот метод обновляет список TurnPriority в CombatPlayerDataInStage и передает иконки персонажей для отображения порядка ходов на всех клиентах. Случайная очередность добавляет элемент непредсказуемости, требуя от игроков адаптации к изменяющимся условиям.

Для оптимизации производительности используется объектный пулинг для визуальных эффектов, таких как индикаторы зон поражения, что снижает нагрузку на систему при частых обновлениях. Событийная система GlobalEventSystem уведомляет клиентов о ключевых изменениях, таких как завершение действия (SendPlayerEndResultTurn) или обновление состояния боя (SendBossActionEnd). Это обеспечивает согласованность визуальных и игровых данных.

Таким образом, сетевая синхронизация игровых состояний в проекте реализована через эффективное сочетание ClientNetworkTransform, NetworkVariable и RPC, обеспечивая стабильный и согласованный кооперативный геймплей. Тестирование показало, что система успешно справляется с синхронизацией данных при подключении четырёх игроков, минимизируя задержки и обеспечивая плавный игровой процесс.

3.1.4. Интерфейс сетевого взаимодействия

Реализация пользовательского интерфейса для сетевого взаимодействия осуществляется через класс `SessionUI`, который управляет визуальными и функциональными аспектами взаимодействия с игровыми сессиями. Этот компонент отвечает за отображение меню поиска сессий, информации о текущем лобби и обработку ввода данных, таких как пароль для защищенных сессий.

Класс `SessionUI` интегрирован с `SessionsManager` и использует перечисление `UIState` (`MainMenu`, `SessionSearch`, `InLobby`) для управления состояниями интерфейса. В состоянии `MainMenu` отображается главное меню, в `SessionSearch` — список доступных сессий, а в `InLobby` — информация о текущем лобби, включая код сессии и пароль (если установлен). Метод `SetUIState` переключает видимость соответствующих панелей (`MainMenu`, `SessionSearchMenu`, `InLobbyMenu`) в зависимости от текущего состояния.

Метод `UpdateSessionList` обновляет список доступных сессий, получаемых через `SessionService.GetSessionsList`. Для каждой сессии создается элемент интерфейса из пула объектов (`sessionPool`), отображающий название сессии, статус приватности (`HasPassword`) и количество игроков (`MaxPlayers - AvailableSlots`). Кнопка присоединения (`joinButton`) вызывает метод `OnJoinSession`, который передает данные сессии в `SessionsManager.JoinSession`.

Для защищенных сессий реализован механизм ввода пароля. Метод `ShowPasswordPrompt` активирует панель ввода пароля (`passwordPromptPanel`), а `HidePasswordPrompt` скрывает ее после успешного подключения или отмены. Кнопка подтверждения (`confirmPasswordButton`) вызывает `JoinSessionWithPassword` из `SessionsManager`, передавая введенный пароль. При неверном пароле метод `ShowErrorMessage` отображает сообщение об ошибке через `errorMessageText`.

Интерфейс также поддерживает функции копирования кода сессии и пароля через метод `CopyText`, связанный с кнопками `copyJoinCodeButton` и `copyPasswordButton`. Кнопка отмены (`canselPasswordButton`) вызывает

CancelPasswordPrompt, сбрасывая ввод и скрывая панель пароля. Метод UpdateLobbyInfo отображает информацию о текущей сессии (код, пароль) только для хоста, активируя панель LobbyInfoPanel и, при необходимости, passwordPanel. UI лобби показан на рисунке 3.1.

Рисунок 3.1 — UI лобби

Данная структура интерфейса обеспечивает интуитивное управление сетевыми функциями, упрощая процесс создания и подключения к сессиям, а также обеспечивая обратную связь для игроков через визуальные элементы и сообщения об ошибках.

3.2 Пошаговая система боя

Пошаговая система боя построена на четырех повторяющихся стадиях: предсказание, выбор действий игроков, выполнение действий игроков и выполнение действий босса. Управление стадиями осуществляется через класс CombatStageManager, а синхронизация между клиентами обеспечивается с использованием Netcode for GameObjects. Механика включает случайную очередность ходов, управление энергией и интерфейс для взаимодействия игроков, что способствует стратегическому планированию и координации.

3.2.1. Архитектура боевой системы

Архитектура пошаговой боевой системы в проекте Few Seconds – Many Deaths реализована через класс CombatStageManager, который выступает центральным компонентом, управляющим всеми стадиями боя. Система построена на модульной структуре, включающей четыре стадии: предсказание (PredictionStage), выбор действий игроков (PlayerTurnStage), выполнение действий игроков (ResultStage) и действия босса (BossTurnStage). Эти стадии хранятся в списке _stages и последовательно активируются, обеспечивая циклический игровой процесс.

Класс `CombatStageManager` инициализируется в методе `OnNetworkSpawn`, где создаются экземпляры всех стадий и вызывается метод `Initialize` для их настройки. Переходы между стадиями управляются методом `TransitionToNextStage`, который использует `NetworkVariable<int>` `currentStageIndexNet` для синхронизации текущей стадии между сервером и клиентами через `Netcode for GameObjects`. На сервере метод `NextStage` активирует следующую стадию, а клиенты обновляют состояние в методе `Update`, отслеживая изменения `currentStageIndexNet`. Класс-диаграмма боевой системы указана на рисунке 3.2.

Рисунок 3.2 — Класс-диаграмма боевой системы

Событийная система `GlobalEventSystem` играет ключевую роль в управлении боем. Событие `PlayerDataUpdated` запускает бой через `StartBattle`, вызывая первую стадию. События `AllPlayersDied` и `BossDied` завершают бой, активируя соответствующие меню (`LoseMenu` или `WinMenu`) и устанавливая флаг `isCombatRunning` в `false`. Каждая стадия наследуется от абстрактного класса `GameState`, реализующего методы `Enter`, `Exit` и `UpdateStage` для управления входом, выходом и обновлением состояния.

Синхронизация между клиентами обеспечивается авторитарной моделью, где сервер управляет критическими событиями, а клиенты получают обновления через `RPC` и сетевые переменные. Флаг `isCombatRunning` контролирует активность боя, предотвращая выполнение стадий после его завершения. Интерфейс, включающий таймер (`timerText`), текст подтверждения (`confirmationText`) и кнопку (`confirmationButton`), интегрирован с системой для отображения текущего состояния боя.

Такая архитектура обеспечивает гибкость, позволяя легко добавлять новые стадии или модифицировать существующие, а также гарантирует стабильную сетевую синхронизацию, что критично для кооперативного геймплея.

Модульность и использование событийной системы упрощают тестирование и дальнейшее развитие боевой механики.

3.2.2. Стадия предсказания

Стадия предсказания, реализованная в классе `PredictionStage`, является первой фазой пошаговой боевой системы в проекте *Few Seconds – Many Deaths*. На этой стадии босс демонстрирует свои будущие атаки, не нанося урона и не накладывая эффекты, что дает игрокам возможность спланировать свои действия. Основная цель стадии — предоставить информацию о предстоящих действиях босса, способствуя стратегическому взаимодействию команды.

При входе в стадию метод `Enter` инициирует выбор комбо босса через `BossManager.ChoiceCombo`, который выполняется на сервере. Это гарантирует, что все клиенты получают согласованные данные о действиях босса. Одновременно восстанавливается энергия игроков с помощью метода `RestoreEnergy`, который добавляет фиксированное количество энергии (`EnergyPerTurn`) к текущему значению каждого игрока, не превышая максимального лимита (`MaxEnergy`). Энергия синхронизируется между клиентами через RPC-вызов `ChangePlayerEnergyRpc`. Учет эффектов, изменяющих энергию, реализован в методе `EnergyUpdate`, который проверяет наличие модификаторов через `EffectsManager`.

Метод `ShieldsRemove` сбрасывает щиты босса и всех игроков, устанавливая значения `CurrentShield` в ноль. Это действие синхронизируется через событие `GlobalEventSystem.SendPlayerShieldChanged`, обновляющее интерфейс на всех клиентах. Событие `BossEndCombo`, вызываемое после завершения анимации комбо босса, инициирует переход к следующей стадии через `CombatStageManager.TransitionToNextStage`.

Визуальная обратная связь обеспечивается через `FeelFeedbacksManager.SetActivePredictionEffect`, активирующий эффект, указывающий на стадию предсказания. События `GlobalEventSystem` (`SendPredictionStageStarted`, `SendPredictionStageEnded`) уведомляют другие

компоненты о начале и завершении стадии, обеспечивая корректное обновление интерфейса и игровой логики. Сетевая синхронизация, реализованная через вызовы на сервере и RPC, гарантирует, что все игроки видят одинаковую последовательность действий босса и обновленные параметры энергии и щитов.

3.2.3. Стадия выбора действий игроков

Стадия выбора действий игроков в проекте Few Seconds – Many Deaths реализована через класс `PlayerTurnStage`, который управляет процессом принятия решений игроками в рамках ограниченного времени. Эта стадия является ключевой для кооперативного геймплея, позволяя каждому игроку одновременно выбирать перемещения или навыки, что требует стратегической координации. Система интегрирована с сетевой моделью `Netcode for GameObjects`, обеспечивая синхронизацию действий между клиентами.

При входе в стадию метод `Enter` инициализирует процесс, активируя пользовательский интерфейс через `EnablePlayerTurnUI`. На сервере запускается корутина `Timer`, которая отслеживает время, доступное для выбора (`maxTurnTime`), используя `NetworkVariable<float> remainingTime` для синхронизации. Оставшееся время отображается в элементе интерфейса `timerText`, обновляемом через `UpdateTimerUIRpc`. Интерфейс, управляемый через `GUIDataBase`, включает кнопки выбора навыков и кнопку подтверждения (`confirmationButton`), которая становится активной для живых игроков (`aliveStatus` из `CombatPlayerDataInStage`).

Игроки подтверждают свои действия, нажимая кнопку подтверждения или используя ввод через `InputActions.Combat.EndPlayerTurn`, что вызывает метод `ConfirmEndTurn`. Этот метод отключает интерфейс выбора (`EndTurnButtonActiveChange`) и отправляет серверу запрос через `ConfirmEndTurnRpc`, увеличивая счетчик подтверждений (`playersConfirmed`, синхронизируемый через `NetworkVariable<int>`). Текущее количество подтверждений отображается в `confirmationText` в формате "текущее/всего живых игроков", обновляемом через `UpdateConfirmationUI`.

Если все живые игроки подтверждают действия (`playersConfirmed.Value >= CountOfAlivePlayers`) или время истекает (`remainingTime.Value <= 0`), сервер вызывает `StartEndingTurn`, который отключает интерфейс, деактивирует ввод (`inputActions.Disable`) и отправляет событие `PlayerTurnEnding` через `GlobalEventSystem`. Это завершает стадию, иницируя переход к следующей через `TransitionToNextStage` из `CombatStageManager`. Для клиентов, не завершивших выбор, действия автоматически подтверждаются по истечении времени.

Стадия завершается событием `PlayerTurnStageEnded`, отправляемым через `GlobalEventSystem`, а интерфейс отключается через `EnablePlayerTurnUI(false)`. Сетевая синхронизация обеспечивает согласованность таймера и подтверждений, а интеграция с `GUIDataBase` гарантирует интуитивное взаимодействие, позволяя игрокам сосредоточиться на тактическом планировании.

3.2.4. Стадия выполнения действий игроков

Стадия выполнения действий игроков реализована через класс `ResultStage`, который отвечает за последовательное выполнение перемещений и навыков, выбранных игроками на предыдущей стадии. Эта стадия обеспечивает синхронизированное воспроизведение действий в порядке, определенном `TurnOrderManager`, с учетом сетевой синхронизации для всех клиентов. Основной задачей является корректное выполнение игровых действий с визуальной обратной связью, поддерживая динамику и последовательность кооперативного боя.

При входе в стадию метод `Enter` иницирует процесс выполнения действий, вызывая `SendStartResultStageForPlayerRpc` с задержкой, заданной параметром `TimeBeforeFirstPlayerMovement`. Это обеспечивает плавное начало стадии, давая клиентам время на обновление интерфейса. Событие `GlobalEventSystem.SendResultStageStarted` уведомляет всех клиентов о начале стадии. На сервере счетчики `PlayerEndMovind` и `PlayerEndResultTurn`,

реализованные как `NetworkVariable`, обнуляются для отслеживания завершенных перемещений и действий.

Процесс выполнения действий делится на два этапа: перемещение и применение навыков. Перемещение персонажей по клеточному полю начинается с вызова `SendStartResultStageForPlayerRpc` для игрока с текущим приоритетом (`_orderInTurnPriority`). После завершения перемещения событие `PlayerEndMoving` увеличивает `PlayerEndMovind` через `ConfirmPlayerEndMovingRpc`, а следующий игрок получает команду на выполнение с задержкой `TimeBetweenPlayerMovement`. Когда все перемещения завершены (`PlayerEndMovind.Value >= playerCount`), сервер вызывает `SendStartCastPlayer` с задержкой `TimeBeforeFirstPlayerCast`, иницируя этап применения навыков.

Применение навыков происходит аналогично: событие `PlayerEndResultTurn` увеличивает `PlayerEndResultTurn` через `ConfirmPlayerEndResultTurnRpc`, а `SendAllPlayersEndMovingRpc` уведомляет клиентов о начале выполнения навыков для следующего игрока. Задержки между действиями обеспечивают визуальную читаемость, позволяя игрокам отслеживать ход боя. Интерфейс обновляется через события `GlobalEventSystem`, такие как `SendStartResultStageForPlayer` и `SendStartCastPlayer`, отображая анимации перемещений и эффекты навыков.

Стадия завершается, когда все игроки выполнили свои действия (`PlayerEndResultTurn.Value >= playerCount`), после чего вызывается `EndTurn`, переводящий бой в стадию действий босса. Метод `Exit` отправляет событие `GlobalEventSystem.SendResultStageEnded`, уведомляя клиентов о завершении стадии. Сетевая синхронизация, реализованная через `RPC` и `NetworkVariable`, гарантирует согласованность порядка и результатов действий между сервером и клиентами, минимизируя рассинхронизацию.

3.2.5. Стадия действий босса

Стадия действий босса в проекте Few Seconds – Many Deaths реализована через класс `BossTurnStage` и является заключительным этапом цикла пошаговой боевой системы. На этой стадии босс выполняет атаки, выбранные на стадии предсказания, нанося урон игрокам и применяя эффекты, что делает её ключевой для динамики боя. Реализация обеспечивает синхронизацию действий между клиентами с использованием `Netcode for GameObjects` и событийной системы `GlobalEventSystem`.

Класс `BossTurnStage` инициализируется в методе `Initialize`, где устанавливается связь с `CombatStageManager` и подписка на событие `BossEndCombo`, которое инициирует завершение стадии. Метод `Enter` запускает выполнение комбо босса через вызов `BossManager.CastCombo`, активируемый на сервере. Это действие сопровождается событием `SendBossTurnStageStarted`, уведомляющим клиентов о начале стадии. Атаки босса, такие как "Круговая атака" или "Энергия меча", реализуются с учетом их параметров (урон, радиус, эффекты), которые могут изменяться в зависимости от фазы боя.

Обновление стадии осуществляется в методе `UpdateStage`, где сервер отслеживает состояние боя, хотя в текущей реализации основная логика сосредоточена в `BossManager`. По завершении комбо босса событие `BossEndCombo` вызывает метод `EndingTurn`, который с задержкой (`TimeBetweenRounds`, установленной в 1 секунду) инициирует переход к следующей стадии через `CombatStageManager.TransitionToNextStage`. Задержка обеспечивает плавность игрового процесса, давая игрокам время воспринять результаты атак.

Выход из стадии происходит через метод `Exit`, который отправляет событие `SendBossTurnStageEnded`, уведомляющее клиентов о завершении. Сетевая синхронизация обеспечивается тем, что основные действия (выбор и выполнение комбо) выполняются на сервере, а клиенты получают обновления через события `GlobalEventSystem`. Это гарантирует согласованность отображения атак и эффектов у всех игроков.

Таким образом, стадия действий босса завершает цикл боя, реализуя атаки с учетом их визуальных и игровых эффектов. Интеграция с сетевой системой и событийным управлением обеспечивает стабильность и динамичность, подчеркивая тактическую важность предшествующих стадий для противодействия боссу.

3.2.6. Управление очередностью ходов

Управление очередностью ходов в проекте реализовано через класс `TurnOrderManager`, который обеспечивает случайное определение порядка действий игроков в каждом боевом цикле. Эта механика добавляет элемент непредсказуемости, требуя от игроков адаптации к динамично меняющимся условиям боя, что усиливает стратегическую составляющую кооперативного геймплея.

Класс `TurnOrderManager` инициализируется в методе `Awake`, где он подписывается на событие `GlobalEventSystem.PredictionStageStarted`, запускающее пересчет порядка ходов в начале стадии предсказания. В методе `Start` иконки персонажей (`playersIconsList`) настраиваются в соответствии с данными из `PlayerInfoData`, отображая спрайты героев (`HeroIcon`) для активных игроков, а неиспользуемые иконки скрываются.

Основная логика определения порядка реализована в методе `RandomiseTurnOrder`, который выполняется только на хосте (`NetworkManager.Singleton.IsHost`). Метод получает текущий список приоритетов (`TurnPriority`) из `CombatPlayerDataInStage` и применяет алгоритм случайного перемешивания (`RandomiseList`). Алгоритм использует `System.Random` для обмена элементов списка, обеспечивая равновероятное распределение порядка. Например, для четырех игроков создается случайная перестановка их ID (0, 1, 2, 3), такая как [2, 0, 3, 1], определяющая последовательность ходов.

Листинг 3.2 — метод случайного перемешивания

```
private void RandomiseList(List<int> list)
```

```

{
    System.Random rng = new System.Random();
    int elementIndex = PlayerInfoData.instance.PlayerCount;
    while (elementIndex > 1)
    {
        elementIndex--;
        int randomElementIndex = rng.Next(elementIndex + 1);
        (list[elementIndex],          list[randomElementIndex])      =
(list[randomElementIndex], list[elementIndex]);
    }
}

```

Синхронизация порядка между клиентами осуществляется через метод `SetNewTurnOrderRpc`, вызываемый с помощью `RPC (SendTo.ClientsAndHost)`. Этот метод передает массив нового порядка (`newTurnOrder`) всем клиентам, обновляя `TurnPriority` в `CombatPlayerDataInStage`. Одновременно обновляется интерфейс: для каждой позиции в `playersIconsList` устанавливается соответствующий спрайт героя из `PlayerInfoData.HeroDataList`, что визуально отражает текущий порядок ходов.

Такая реализация обеспечивает корректную синхронизацию очередности в сетевой игре, минимизируя расхождения между клиентами. Случайный порядок, обновляемый в начале каждого цикла, стимулирует игроков координировать действия, учитывая возможные изменения в последовательности, и повышает реиграбельность боевых сценариев.

3.3 Перемещение и навыки персонажей

Эти механики обеспечивают тактическое взаимодействие игроков с игровым полем и противником, поддерживая кооперативный геймплей. Перемещение реализовано через алгоритм поиска пути и управление энергией, а система навыков использует `ScriptableObject` для гибкой настройки.

Визуализация игрового поля, путей и зон действия навыков осуществляется через пулы объектов и динамическое обновление интерфейса. Сетевая синхронизация обеспечивается с помощью Netcode for GameObjects, гарантируя согласованность действий между клиентами.

3.3.1 Система перемещения персонажей

Система перемещения персонажей в Few Seconds – Many Deaths реализована через класс `PlayerMovementController`, обеспечивающий тактическое перемещение по клеточному полю в пошаговом режиме. Механика позволяет игрокам планировать маршруты, учитывая затраты энергии и препятствия, с полной сетевой синхронизацией для кооперативного геймплея. Перемещение интегрировано с системой боя, активируясь на стадии выбора действий (`PlayerTurnStage`), и визуализируется через `PathVisualizer`.

Класс `PlayerMovementController` инициализируется в методе `Awake`, устанавливая синглтон (`instance`), и в `OnNetworkSpawn`, где подключаются события `GlobalEventSystem` (`PlayerTurnStageStarted`, `PlayerTurnEnding`, `StartResultStageForPlayer`) и действия ввода через `InputActions` (`SelectTile`, `CancelAction`). Перемещение начинается в методе `StartDefineMovement`, активирующем ввод для живых и не оглушенных игроков (`isAlive`, `!isStunned`) и сохраняющем начальную позицию (`LastPosition`).

Основной метод `AddNewPointsToList` обрабатывает выбор клетки по координатам мыши, преобразуя их в позицию на поле (`GameplayTilemap.WorldToCell`). Проверяется доступность клетки (`MapClass.IsPlayable`) и строится маршрут с помощью `CreatePathRoute`, использующего алгоритм поиска пути (`GridPathfinding`). Для каждой точки пути метод `DefineMovement` определяет тип перемещения: горизонтальное/вертикальное (`HorVer`, 2 энергии), диагональное (`Diagonal`, 3 энергии), неподходящее (`TooFar`) или отсутствие движения (`None`). Если энергии достаточно, точка добавляется в `MovementList`, обновляется `LastPosition`, а энергия уменьшается через `ChangePlayerEnergyRpc`. Звуковые эффекты

воспроизводятся через `FXManager.PlayMovementSFX` или `PlayLackOfEnergySFX` при нехватке энергии. Событие `SendEnergyUsedOnMovement` уведомляет о затратах.

Листинг 3.3 — метод определения типа перемещения

```
private void DefineMovement(Vector2 PlayerCoor, Vector2 targetPoint)
{
    if (PlayerCoor.x == targetPoint.x && PlayerCoor.y == targetPoint.y)
        MovementIndex = Movement.None;
    else if (Math.Abs(PlayerCoor.x - targetPoint.x) > 1 || Math.Abs(PlayerCoor.y - targetPoint.y) > 1)
        MovementIndex = Movement.TooFar;
    else if (PlayerCoor.x == targetPoint.x || PlayerCoor.y == targetPoint.y)
        MovementIndex = Movement.HorVer;
    else
        MovementIndex = Movement.Diagonal;
}
```

Отмена перемещения реализована в `RemoveLastPointsInList`, удаляющем точки до последнего чекпоинта (`MovementListCheckPoints`) и восстанавливающим энергию через `ChangePlayerEnergyRpc`. События `SendEnergyRestored` и `SendPathChanged` обновляют интерфейс, а `FXManager.PlayActionCanceledSFX` воспроизводит звук отмены. Подтверждение пути происходит в `ApproveThePath`, отключая ввод, а выполнение — в `StartMoving`, вызываемом для игрока в порядке очереди (`turnPriority`). Метод `CorrectingPath` исключает клетки, занятые героями или боссом, проверяя через `MapClass.GetMapObjectList`.

Корутина `MovePlayer` последовательно перемещает персонажа по точкам `MovementList`, обновляя позицию через `transform.position` и синхронизируя координаты (`ChangePlayerCoordinatesRpc`) и состояние поля (`ChangeMapStatesRpc`). Задержка между шагами (`timeBetweenMovement`) обеспечивает плавность. После завершения `MovementList` очищается, и вызывается `SendPlayerEndMoving`. Сетевая синхронизация осуществляется через

RPC (SendPlayerStartMoveRpc, SendPlayerEndMoveRpc), гарантируя согласованность позиций и событий между клиентами.

Эта система обеспечивает интуитивное управление перемещением, интегрированное с интерфейсом и сетевой логикой, позволяя игрокам стратегически планировать действия с учетом энергии и препятствий.

3.3.2 Управление игровым полем

Управление игровым полем в проекте Few Seconds – Many Deaths реализовано через класс MapClass, который обеспечивает структуру и логику клеточного поля, используемого для перемещения персонажей, размещения объектов и проверки их состояния. Поле является основой тактического геймплея, позволяя игрокам и боссу взаимодействовать с окружающей средой. Реализация включает инициализацию клеток, управление их состоянием, проверку проходимости и интеграцию с алгоритмом поиска пути, что гарантирует корректное функционирование механик перемещения и навыков.

Класс MapClass инициализируется в методе Awake, где создается двумерный массив TilesInfoArray размером $Max_A \times Max_B$, содержащий информацию о каждой клетке (доступность, объекты, эффекты). Свойство GameplayTilemap предоставляет доступ к тайловой карте Unity, а tileZero определяет смещение координат для корректного преобразования между мировыми и локальными координатами. Метод FindTerrain, вызываемый в Start, сканирует поле, определяя проходимые клетки: если клетка в GameplayTilemap отсутствует, она помечается как непроходимая (NoPlayableTile), а проходимые клетки добавляются в список AllTiles. Событие SendMapClassInitialized уведомляет другие системы об успешной инициализации.

Состояние клеток управляется через методы, реализующие интерфейс IMap. Метод SetHero добавляет объект Hero с указанным playerID в список MapObjectList клетки, а SetBoss и SetCorpse выполняют аналогичные действия для босса и трупов. Методы удаления (RemoveHero, RemoveBoss, RemoveCorpse) очищают соответствующие объекты из списка. Для временной блокировки

клеток используется `SetTempBloked`, добавляющий объект `TempBloked`, а `RemoveTempBloked` снимает блокировку. Метод `UpdateWalkability` обновляет проходимость клетки в алгоритме поиска пути (`GridPathfinding`), обеспечивая актуальность маршрутов.

Проверка состояния клеток осуществляется через `IsPlayable`, который возвращает `true`, если клетка не содержит `NoPlayableTile` и находится в пределах поля (`CheckBoundaries`). Метод `GetMapObjectList` возвращает список объектов в указанной клетке, а `MapObjectCheck` проверяет объекты в заданной области, что используется для определения целей навыков. При смерти игрока событие `PlayerDied` вызывает `RemoveHero` и `SetCorpse`, обновляя поле для отображения трупа.

Интеграция с другими системами, такими как `PlayerMovementController` и `PlayerSkillManager`, осуществляется через методы доступа к данным поля. Например, `IsPlayable` используется для проверки возможности перемещения, а `GetMapObjectList` — для предотвращения наложения персонажей. Сетевая синхронизация изменений состояния поля обеспечивается через RPC, вызываемые в `PlayerMovementController` (`ChangeMapStatesRpc`), что гарантирует согласованность данных между клиентами. Эта реализация обеспечивает гибкое и надежное управление игровым полем, поддерживая динамичный и тактический геймплей.

3.3.3. Система навыков персонажей

Система навыков персонажей в игре *Few Seconds – Many Deaths*, реализованная в классе `PlayerSkillManager`, обеспечивает игрокам возможность использовать тактические способности в стадии выбора действий (`PlayerTurnStage`). Навыки, настроенные через `SkillScript` на основе `ScriptableObject`, предоставляют гибкость в определении эффектов, зон действия и затрат энергии. Система поддерживает кооперативный геймплей благодаря сетевой синхронизации через `Netcode for GameObjects`, гарантируя согласованность действий между клиентами.

Выбор навыка осуществляется через метод `GetSkill`, активируемый нажатием клавиш (`SelectSkillByButton`) или интерфейсом. Перед использованием проверяются доступность энергии игрока (хранится в `CombatPlayerDataInStage`) и перезарядка навыка, управляемая классом `SkillCooldownManager`. Если условия соблюдены, навык добавляется в список выбранных (`SkillList`), а энергия вычитается через `ChangePlayerEnergyRpc`. Игрок указывает клетки-цели мышью, которые сохраняются в `TargetTileList` методом `AddSkillToList`. Метод `CanselAction` позволяет отменить выбор, возвращая энергию и сбрасывая перезарядку через `SendChangeSkillCooldownRpc`.

Выполнение навыков происходит в стадии результатов (`StartSkillSystem`), где метод `CastAction` последовательно активирует каждый навык из `SkillList`. Сетевая синхронизация осуществляется через `CastActionRpc`, который вызывает метод `Cast` объекта `SkillScript`, передавая позиции игрока, цели и идентификатор игрока. Это обеспечивает одновременное отображение эффектов (например, урона или лечения) для всех игроков. Перезарядка навыков уменьшается на стадии предсказания методом `SkillCooldownDecrease`, обновляя состояние через событие `SendUpdateCooldown`.

Листинг 3.4 — метод сетевого вызова выполнения навыка

```
[Rpc(SendTo.ClientsAndHost)]

private void CastActionRpc(int casterPlayerId, int skillNumber, Vector2
CasterPosition, Vector2 ActualCasterPosition, Vector2[] TargetPoints, int skillIndex)
{
    this.casterPlayerId = casterPlayerId;

    int variation = PlayerInfoData.instance.SkillChoiceList[casterPlayerId].
variationList[skillNumber];

    SkillScript skillScript =
PlayerInfoData.instance.HeroDataList[casterPlayerId].
SkillList[skillNumber].SkillVariationsList[variation].SkillScript;

    CurrentSkill = skillScript;
```

```

        skillScript.Cast(CasterPosition, ActualCasterPosition, TargetPoints,
casterPlayerId, skillIndex);
    }

```

Система навыков интегрирована с визуализацией (SkillVisualizer), отображающей доступные и затронутые зоны действия, что помогает игрокам принимать тактические решения. Гибкость SkillScript позволяет легко добавлять новые навыки, а сетевая синхронизация обеспечивает согласованность в кооперативной игре, усиливая взаимодействие между игроками.

3.3.4 Визуализация игрового поля и действий

Визуализация игрового поля и действий в Few Seconds – Many Deaths реализована через класс GridVisualizer, объединяющий компоненты TileManager, PathVisualizer, SkillVisualizer и BossTileManager. Эти компоненты обеспечивают динамическое отображение клеток, путей перемещения, зон действия навыков и позиции босса, помогая игрокам принимать тактические решения в кооперативной игре. Для оптимизации производительности используются пулы объектов, минимизирующие создание и удаление элементов интерфейса. События, такие как PathChanged и PlayerSkillChoosed, обеспечивают реактивное обновление визуальных элементов в реальном времени.

TileManager отвечает за отображение клеток, занятых героями, и селектора текущей клетки. Метод CreatePlayersTiles создаёт визуальные элементы для героев с использованием цветовой палитры игрока (PlayerInfoData.ColorList), а методы ShowPlayersTiles и HidePlayersTiles управляют их видимостью в зависимости от стадии игры. Селектор клетки, обновляемый в UpdateTileSelector, меняет вид в зависимости от содержимого клетки: союзник (Hero), враг (Boss), препятствие (TempBloked) или стандартная клетка. Это позволяет игрокам быстро оценивать состояние поля.

PathVisualizer отображает маршруты перемещения персонажей. Метод UpdatePath строит временный путь от текущей позиции игрока (LastPosition) до

выбранной клетки, учитывая доступную энергию. Подтверждённые пути визуализируются через `CreateActualPath`, используя пулы объектов (`playerPathTilePool`, `playerPathEndTilePool`) для плиток пути и конечной точки. Метод `SendPathTiles` синхронизирует отображение путей между клиентами через `SendPlayerPathTileRpc`, применяя цвет игрока для различия маршрутов в кооперативной игре.

`SkillVisualizer` отвечает за визуализацию зон действия навыков. Метод `UpdateSkillAvaliableArea` отображает доступные для выбора клетки, рассчитанные в `SkillScript.AvailableTiles`, а `UpdateSkillAffectedArea` показывает область поражения навыка на основе текущей позиции курсора. Подтверждённые зоны отображаются через `UpdateAprovedAffectedAreas` с использованием пула `skillAprovedAffectedTilePool`. Пулы объектов (`skillAvaliableTilePool`, `skillAffectedTilePool`) оптимизируют производительность, особенно при частом обновлении зон. Очистка зон происходит через методы `ClearAvaliableArea` и `ClearAprovedAffectedAreas` при смене действий или стадии.

`BossTileManager` управляет отображением клетки босса. Метод `CreateBossTile` создаёт визуальный элемент на стартовой позиции босса (`BossManager.CurrentCoordinates`), а методы `ShowBossTile` и `HideBossTile` синхронизируют его видимость с перемещениями босса. Это обеспечивает чёткое представление о положении противника на поле, что критично для планирования действий в кооперативной игре.

Визуализация тесно связана с геймплеем, предоставляя игрокам обратную связь о возможных действиях. События `GlobalEventSystem` (например, `PathChanged`, `PlayerSkillAproved`) обеспечивают динамическое обновление интерфейса, а сетевая синхронизация через `Netcode for GameObjects` гарантирует согласованность отображения для всех игроков. Использование пулов объектов и оптимизированных методов визуализации позволяет поддерживать высокую производительность даже при интенсивном взаимодействии с игровым полем.

3.3.5. Архитектура боевой системы

Действия босса в игре Few Seconds – Many Deaths, реализованные через класс `BossManager`, обеспечивают динамичное противодействие игрокам, усиливая тактический аспект кооперативного геймплея. Босс выполняет комбинации атак, выбираемые случайным образом на стадии предсказания, с последующей синхронизацией через `Netcode for GameObjects`. Визуализация действий включает отображение призрачного спрайта босса и поворот к ближайшему игроку, что помогает игрокам прогнозировать угрозы и планировать защиту.

Выбор комбинации атак осуществляется методом `ChoiceCombo`, который случайным образом определяет комбо из списка (`ComboAttackList`) для текущей фазы боя (`CurrentAct`). Метод `GetTargetPointsForActions` задаёт координаты целей для каждой атаки в комбо, синхронизируя их через `GetTargetPointsForActionsRpc`. Выполнение атак происходит в методе `CastCombo`, вызывающем `CastActionRpc` для каждой атаки, что обеспечивает применение эффектов (например, урона или блокировки клеток) с учётом целей (`TargetPointsForActions`). Сетевая синхронизация гарантирует, что все игроки видят действия босса в реальном времени.

Визуализация реализована через призрачный спрайт (`GhostBossGameObject`), отображаемый на стадии предсказания методом `EnableGhostBoss`. Это позволяет игрокам видеть предполагаемые действия босса, такие как зоны атаки, и корректировать свои перемещения или навыки. Метод `RotateBossToNearestPlayerRpc` поворачивает спрайт босса (или его призрака) к ближайшему игроку, усиливая визуальную обратную связь. Позиция босса обновляется через `ResetCurrentCoordinatesRpc`, синхронизируя её с `CurrentCoordinates`.

Переход между фазами боя происходит в методе `ChangeAct`, активируемом при достижении нуля здоровья (`CurrentHPInCurrentAct`). Новая фаза обновляет характеристики босса (например, максимальное здоровье) через `ChangeArcRpc`, а при исчерпании всех фаз вызывается событие `BossDied`. Метод `StopCombo` скрывает призрачный спрайт на стадии игроков, завершая цикл действий босса.

Механика действий босса влияет на тактику игроков, вынуждая их избегать зон атаки и координировать действия для минимизации урона. Сетевая синхронизация и визуализация обеспечивают прозрачность и согласованность в кооперативной игре, делая босса ключевым элементом стратегического взаимодействия.

3.3 Перемещение и навыки персонажей

Навыки игроков и действия босса используют гибкую архитектуру на основе ScriptableObject, позволяющую настраивать эффекты, визуализацию и звуковое сопровождение. Классы SkillScript и BossActionScript предоставляют базовую функциональность, а их наследники реализуют конкретные механики, такие как нанесение урона, наложение щитов или перемещение. События анимаций управляются через AnimationEventManager, синхронизируя выполнение действий с визуальными эффектами. Сетевая синхронизация через Netcode for GameObjects обеспечивает согласованность в многопользовательской среде.

3.4.1 Базовая структура навыков игроков

Класс SkillScript является базовым для всех навыков игроков в игре Few Seconds – Many Deaths, обеспечивая гибкую настройку тактических действий через архитектуру ScriptableObject. Он определяет ключевые параметры: стоимость энергии (EnergyCost), время перезарядки (SkillCooldown), количество целей (TargetCount) и флаг возможности перемещения (IsMovable). Эти параметры позволяют настраивать разнообразные эффекты, такие как атака, защита или перемещение, поддерживая стратегическое взаимодействие в кооперативном геймплее.

Основной метод Cast управляет процессом применения навыка, последовательно вызывая CastStart для инициализации параметров (позиция

игрока, цели), CastFX для воспроизведения визуальных и звуковых эффектов (CastVFXPrefab, AreaVFXPrefab, castSFX) и CastEnd для завершения действия с вызовом события SendPlayerSkillEnd. Метод CastStart учитывает возможность перемещения, корректируя координаты целей через ChangeSelectedCellCoordinate, если игрок изменил позицию. Методы Area и AvailableTiles определяют зону действия навыка и доступные для выбора клетки, обеспечивая точное нацеливание.

Визуализация навыков оптимизирована с использованием пулов объектов из SkillTilePoolManager (например, attackTilePool, shieldTilePool), что минимизирует создание и удаление объектов. Метод SpawnSkillObjects размещает тайлы эффектов (TilePrefab) в зоне действия, а HideSkillObjects возвращает их в пул для повторного использования. Поддержка локализации реализована через GetLocalizeVariablesList, позволяя адаптировать текстовые описания навыков.

Взаимодействие с игровыми объектами осуществляется через методы GetAffectedCombatObjectList и GetHeroCombatObject, которые возвращают списки затронутых сущностей (Hero, Boss) для применения эффектов, таких как урон или щиты. Такая структура позволяет легко расширять систему новыми навыками, сохраняя производительность и тактическую глубину.

3.4.2 Примеры реализации навыков игроков

Навыки игроков в игре Few Seconds – Many Deaths реализуются через наследование от базового класса SkillScript, что позволяет создавать разнообразные механики с гибкой настройкой эффектов, визуализации и звукового сопровождения. Рассмотрим два примера: HorizontalSign и QuickShot, демонстрирующие различные подходы к реализации тактических действий, их визуализации и взаимодействия с игровой средой.

Навык HorizontalSign создаёт горизонтальную линию действия, накладывая щит на союзников в зоне. Метод Area формирует область действия (CoordinateLine), охватывающую все клетки в строке от левого края карты до

правого (MapClass.Max_A). Навык активируется через метод Cast, который вызывает CastStart для инициализации координат, StartAnimation для воспроизведения анимации персонажа ("Cast2") и CastFX для отображения визуальных эффектов (CastVFXPrefab, AreaVFXPrefab) и звуков (castSFX). Метод ApplyShield накладывает щит (shieldValue) на объекты в зоне действия, определённые через GetAffectedCombatObjectList. Визуализация зоны осуществляется с использованием пула объектов (shieldTilePool) в методе SpawnSkillObjects, а метод HideSkillObjects возвращает тайлы в пул, оптимизируя производительность. Навык не зависит от боеприпасов или дополнительных условий, что делает его надёжным защитным инструментом.

Навык QuickShot реализует быструю атаку по одной клетке с учётом боеприпасов и бонусного урона. Метод Area возвращает одиночную цель (selectedCellCoordinate), а AvailableTiles предоставляет все доступные клетки карты. В методе Cast вызываются CastStart, StartAnimation (анимация "Cast1_Part1" или "Cast1_Part2" в зависимости от порядка навыков) и CastFX для эффектов и звука (castSFX). Метод CastQuickShot проверяет наличие боеприпасов через ShotsManager и, при их наличии, запускает корутину ShotMovement, анимирующую траекторию снаряда (QuickShotPrefab) с заданной скоростью (shotSpeed). Урон (Damage) увеличивается на 50% при наличии союзников рядом (IsPlayersNear), а метод ApplyDamage наносит урон и применяет эффект кровотечения (bleedEffect) через CombatMethods. Визуализация включает эффект попадания (HitVFXPrefab) и тайлы атаки (attackTilePool). При отсутствии боеприпасов отображается сообщение (noAmmoLocalizedString), а тайлы очищаются (HideSkillObjects).

Оба навыка используют пулы объектов из SkillTilePoolManager для оптимизации, минимизируя создание и удаление визуальных элементов. Звуковые эффекты (castSFX, hitSFX) и анимации усиливают обратную связь, помогая игрокам отслеживать действия. HorizontalSign ориентирован на командную защиту, тогда как QuickShot подходит для точечных атак с тактическими решениями, зависящими от боеприпасов и позиционирования.

3.4.3 Базовая структура действий босса

Класс `BossActionScript` служит основой для реализации действий босса в игре *Few Seconds – Many Deaths*, обеспечивая гибкую настройку атак и эффектов через архитектуру `ScriptableObject`. Он определяет базовые компоненты для визуализации и звукового сопровождения, а также предоставляет методы для выполнения действий и взаимодействия с игровыми объектами. Действия босса синхронизируются через `Netcode for GameObjects`, гарантируя согласованность в кооперативной игре, а оптимизация достигается за счёт использования пулов объектов.

Класс включает параметры для визуальных эффектов (`affectedTile` для отображения зон поражения, `hitVFX` для эффектов попадания) и звуков (`castSFX` для звука активации, `hitSFX` для звука попадания). Метод `Cast` инициирует действие, вызывая `CastStart` для настройки параметров (цели в `TargetPoints`, текущая фаза боя в `act`), `CastAreaForSkill` для отображения зоны поражения и `CastEnd` для завершения, отправляя событие `BossActionEnd`. Метод `GetCastPoint` определяет координаты целей, зависящие от фазы боя, что позволяет адаптировать действия босса к прогрессу игры.

Визуализация зон поражения адаптируется к стадии игры: на `PlayerTurnStage` тайлы отображаются с пониженной прозрачностью (`alphaForGhost`), сигнализируя игрокам о предстоящей атаке, а на `BossTurnStage` — с полной. Метод `CastAreaForSkill` использует пул объектов (`SkillTilePoolManager.attackTilePool`) для оптимизации, минимизируя создание новых тайлов. Метод `DestroyAffectedTilesPrefabs` очищает неактивные тайлы, снижая нагрузку на производительность.

Для взаимодействия с игровыми объектами метод `GetAffectedCombatObjectList` возвращает список затронутых сущностей (`Hero`, `Boss`) в указанной зоне, а `DamageEveryOneInTiles` наносит урон всем объектам в клетках, сопровождая это звуковыми и визуальными эффектами. Метод `GetPoint` обеспечивает точное позиционирование эффектов (например, `hitVFX`)

относительно центра объекта. Метод `CastPartOfAction` поддерживает многоэтапные действия, позволяя разбивать сложные атаки на последовательные шаги, синхронизированные с анимациями через `StartAnimation`.

`BossActionScript` обеспечивает модульность и расширяемость, позволяя создавать разнообразные действия босса (например, атаки по площади или рывки) с минимальными изменениями кода.

3.4.4 Примеры реализации действий босса

Действия босса в игре *Few Seconds – Many Deaths* реализуются через наследников класса `BossActionScript`, обеспечивая разнообразные тактические угрозы для игроков. Рассматриваются два примера: `BerserkCircularSlice_Berserk` и `Dash_Berserk`, демонстрирующие нанесение урона, перемещение и взаимодействие с игровым полем. Эти действия используют анимации, звуковые эффекты и визуализацию зон поражения, синхронизируемые через `Netcode for GameObjects` для согласованности в кооперативной игре.

Класс `BerserkCircularSlice_Berserk` реализует круговую атаку босса, создающую зону поражения с радиусом, зависящим от фазы боя (`act`). Метод `GetCastPoint` возвращает центр атаки (координаты босса), а метод `Cast` иницирует действие, вызывая `CastCircularSlice` через `CastPartOfAction` с индексом 1. Зона поражения формируется методом `SquareAOE`, отображаемым через `CastAreaForSkill` с использованием пула `attackTilePool`. Урон наносится игрокам в зоне (`DamagePlayers`), а босс восстанавливает здоровье (`ApplyHeal`) пропорционально нанесённому урону (`healModifier`). Визуализация включает анимацию (`StartAnimation`, триггер "Circle") и звуковые эффекты (`castSFX`, `healSFX`), с прозрачностью тайлов, адаптированной для стадии предсказания (`alphaForGhost`).

Листинг 3.5 — метод исполнения круговой атаки босса

```
private void CastCircularSlice()
```

```

{
    int _radius = radius;
    if (act > 1) _radius = radiusUpgrade + radius;
    List<Vector2> CircularList = GridAreaMethods.SquareAOE
(bossManager.CurrentCoordinates, bossManager.CurrentCoordinates, _radius, true);
    CastAreaForSkill(CircularList);
    DamagePlayers(CircularList);
}

```

Класс Dash_Berserk реализует рывок босса через последовательность случайных клеток, заданных методом GetCastPoint. В фазе боя выше первой выбираются случайные клетки (randomCellCount), а последняя цель — точка рядом с игроком (GetRandomPointNearRandomPlayer). Метод Cast запускает корутину Dash, которая строит пути (GridPathfinding.FindPath) и корректирует их (CorrectingPath), избегая коллизий с героями или боссом. Движение анимируется через MoveAlongPath с убывающей скоростью (StartDashSpeed, SpeedDecreaseMultiplayer), нанося урон (DamageEveryOneInTiles) на каждой конечной клетке в стадии BossTurnStage. Визуализация включает анимацию рывка (StartDash), звуковые эффекты (castSFX, hitSFX) и поворот спрайта (RotateBossToMovementDirection). Позиция босса обновляется через MapClass.SetBoss, синхронизируя координаты (CurrentCoordinates).

Оба действия усиливают тактический аспект, вынуждая игроков предсказывать зоны поражения и координировать перемещения.

3.4.5 Управление событиями анимаций

Управление событиями анимаций в проекте Few Seconds – Many Deaths реализовано через класс AnimationEventManager, обеспечивающий синхронизацию визуальных эффектов с выполнением навыков игроков и действий босса. Этот класс играет ключевую роль в согласовании анимаций с

игровыми событиями, что усиливает визуальную обратную связь и поддерживает динамику кооперативного геймплея.

Метод `SetSkillIndexAndStartActions` управляет анимациями навыков игроков, вызывая метод `CastPartOfSkill` текущего навыка (`CurrentSkill`) из `PlayerSkillManager`. Этот метод позволяет запускать определённые этапы анимации, например, завершение действия в навыке `QuickShot`, где анимация делится на части (`Cast1_Part1`, `Cast1_Part2`, `Cast1_Part3`) для точного соответствия игровым эффектам, таким как выстрел или нанесение урона. Проверка на наличие текущего навыка с помощью `Debug.LogError` предотвращает ошибки при отсутствии активного навыка, обеспечивая надёжность системы.

Метод `SetActionIndexAndStartActions` отвечает за анимации действий босса, вызывая `CastPartOfAction` для текущего действия из списка `BossActionList` в `BossManager`. Например, в `BerserkCircularSlice_Berserk` метод запускает этап анимации круговой атаки (`Circle`), а в `Dash_Berserk` — этапы рывка (`BerserkDash_Loop`). Метод учитывает индекс текущего действия (`CurrentAction`) и проверяет корректность комбо, логируя ошибки (`Debug.LogError`) при несоответствии, что упрощает отладку.

Точное соответствие анимаций игровым событиям, таким как нанесение урона или перемещение, повышает восприятие тактических решений, делая игровой процесс более интуитивным и зрелищным.

3.4.6 Сетевая синхронизация и оптимизация

Сетевая синхронизация навыков игроков и действий босса в игре *Few Seconds – Many Deaths* реализована с использованием `Netcode for GameObjects`, обеспечивая согласованность игровых событий в кооперативной многопользовательской среде. Оптимизация производительности достигается за счёт применения пулов объектов и эффективного управления ресурсами, минимизируя нагрузку на клиентские устройства.

Для навыков игроков сетевая синхронизация осуществляется через метод `CastActionRpc` в классе `PlayerSkillManager`. Этот метод передаёт параметры

навыка, включая идентификатор игрока (`casterPlayerId`), номер навыка (`skillNumber`), позицию кастера (`CasterPosition`), актуальную позицию (`ActualCasterPosition`), координаты целей (`TargetPoints`) и индекс навыка (`skillIndex`). Вызов `CastActionRpc` активирует метод `Cast` соответствующего объекта `SkillScript`, применяя эффекты (например, урон в `QuickShot` или щит в `HorizontalSign`) на всех клиентах. Это гарантирует, что визуальные и игровые эффекты навыков, такие как анимации и изменения состояния объектов, отображаются синхронно для всех игроков.

Действия босса синхронизируются через методы `CastActionRpc` и `GetTargetPointsForActionsRpc` в классе `BossManager`. Метод `GetTargetPointsForActionsRpc` передаёт координаты целей (`TargetPoints`) для каждой атаки в комбо, обеспечивая их отображение на стадии предсказания (`PlayerTurnStage`) через призренный спрайт (`GhostBossGameObject`). Метод `CastActionRpc` вызывает `Cast` объекта `BossActionScript`, применяя эффекты (например, урон в `BerserkCircularSlice_Berserk` или перемещение в `Dash_Berserk`) с учётом текущей фазы боя (`act`) и целей. События `SendBossActionEnd` и `SendTargetPointsForActionsChoosed` завершают синхронизацию, уведомляя клиентов о выполнении действий.

Оптимизация визуализации достигается за счёт использования пулов объектов, реализованных в `SkillTilePoolManager`. Для навыков игроков пулы (`attackTilePool`, `shieldTilePool`, `healTilePool`, `teleportTilePool`) предоставляют тайлы для отображения зон действия и эффектов, минимизируя создание и удаление объектов. Метод `HideSkillObjects` возвращает неактивные тайлы в пул, как видно в `HorizontalSign` и `QuickShot`. Для действий босса метод `DestroyAffectedTilesPrefabs` очищает тайлы атаки (`affectedTile`) после завершения действия, как в `BerserkCircularSlice_Berserk`. Это снижает нагрузку на память и процессор, особенно при частом применении навыков и атак.

Дополнительная оптимизация реализована через адаптивную визуализацию зон действия. В `BossActionScript` метод `CastAreaForSkill` изменяет прозрачность тайлов (`alphaForGhost`) на стадии предсказания, снижая визуальную

нагрузку. В SkillScript методы SpawnSkillObjects и CastVFX проверяют наличие тайлов на карте (GameplayTilemap.HasTile), предотвращая создание объектов на непроходимых клетках. Сетевая синхронизация минимизирует избыточные вызовы RPC, передавая только необходимые данные, такие как координаты и индексы.

Сочетание сетевой синхронизации и оптимизации обеспечивает плавный кооперативный геймплей, позволяя игрокам координировать действия против босса. Эффективное управление ресурсами поддерживает стабильную производительность даже при интенсивных игровых событиях, таких как множественные атаки или массовые эффекты навыков.